Ecole Supérieure
d'Informatique et du Numérique
COLLEGE OF ENGINEERING & ARCHITECTURE

COMPTE RENDU

Programmation Parallèle — Challenge 2 — Génération parallèle de clés RSA avec MPI
4e année Cybersécurité (Ing4-CYB/CCV) - École Supérieure d'Informatique et du Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiant : HATHOUTI Mohammed Taha

Filière : Cybersécurité

Année : 2026/2027

Enseignant : M. BAKHOUYA

Date : 15 octobre 2026

Table des matières

Rappel du sujet **2**

1 Partie II : génération parallèle de clés avec MPI **3**
- 1.1 Organisation du code . . . 3
- 1.2 Distribution des tâches . . . 3
- 1.3 Collecte des résultats . . . 3
- 1.4 Vérification . . . 4
- 1.5 Résultats . . . 4

2 Partie III : évaluation des performances **5**
- 2.1 Protocole de mesure . . . 5
- 2.2 Résultats . . . 5
- 2.3 Analyse . . . 7

Rappel du sujet

Le sujet fournit une implémentation séquentielle de la génération de clés RSA (`rsa.c`, `rsa.h`). La fonction `generate_rsa_keys()` prend deux nombres premiers distincts p et q et calcule

$$N = p \cdot q, \qquad \varphi(N) = (p-1)(q-1),$$

puis l'exposant public e et l'exposant privé d tels que

$$\gcd(e, \varphi(N)) = 1 \qquad \text{et} \qquad e \cdot d \equiv 1 \pmod{\varphi(N)}.$$

Cette implémentation est réutilisée telle quelle (logique et noms inchangés). Seize paires (p, q) sont données (tableau 1 de l'énoncé) ; chaque génération de clés est une tâche indépendante des autres. Le sujet demande :

- **Partie II** : un programme MPI `rsa_mpi.c` qui répartit les 16 tâches entre les processus (chaque tâche exécutée exactement une fois), collecte les résultats (N, e, d) au niveau du maître, vérifie $\gcd(e, \varphi) = 1$ et $(e \cdot d) \bmod \varphi = 1$, et vérifie que le résultat est identique à celui de la version séquentielle ;
- **Partie III** : l'évaluation des performances pour différents nombres de processus (temps, accélération $S(P) = T_1/T_P$, efficacité $E(P) = T_1/(P \cdot T_P)$), avec une analyse tenant compte du nombre de tâches, de la répartition de la charge, de l'initialisation des processus et du surcoût de communication. Les tâches étant très courtes, la charge doit être répétée pour obtenir des temps mesurables.

1 Partie II : génération parallèle de clés avec MPI

1.1 Organisation du code

Conformément au sujet, l'implémentation RSA est séparée du programme MPI :

- `rsa.c` / `rsa.h` : fonctions RSA fournies (`gcd`, `extended_gcd`, `modular_inverse`, `is_prime`, `generate_rsa_keys`), compilées avec `cc` ;
- `rsa_mpi.c` : programme MPI (`main`, distribution, collecte, vérification), compilé avec `mpicc` ;
- `bench_mpi.c` : programme de mesure de la partie III, séparé de `rsa_mpi.c` ;
- un `Makefile` qui compile les deux exécutables, supprime les fichiers objets après l'édition de liens et fournit les cibles `run`, `run_all`, `bench` et `graphs`.

1.2 Distribution des tâches

Les 16 paires sont stockées dans un tableau constant, présent dans chaque processus (la mémoire n'est pas partagée en MPI, chaque processus est une copie du programme). La répartition se fait uniquement à partir de `rang` et `taille` :

```
/* Distribution cyclique : la tache i est traitee par le processus
   (i % taille) */
void calculer_taches(int rang, int taille, long long local[
   NB_TACHES][NB_CHAMPS]) {
    for (int i = rang; i < NB_TACHES; i += taille) {
        long long N = 0;
        long long e = 0;
        long long d = 0;
        int retour = generate_rsa_keys(premiers[i][0], premiers[i
           ][1], &N, &e, &d);

        local[i][CH_N] = N;
        local[i][CH_E] = e;
        local[i][CH_D] = d;
        local[i][CH_EXECUTEE] = 1;
        local[i][CH_SUCCES] = (retour == 0);
    }
}
```

Le processus `rang` traite les tâches $\texttt{rang}, \texttt{rang} + \texttt{taille}, \texttt{rang} + 2\,\texttt{taille}, \ldots$: les ensembles de tâches sont disjoints et leur union couvre les 16 tâches, donc chaque tâche est exécutée par *exactement un* processus. Si le nombre de processus dépasse 16, les processus en trop n'ont aucune tâche et restent inactifs. Chaque processus range ses résultats (N, e, d) dans sa copie locale du tableau, à la ligne de la tâche.

1.3 Collecte des résultats

Chaque ligne du tableau local n'est remplie que par un seul processus, les autres lignes valent 0. Un seul `MPI_Reduce` avec l'opération `MPI_SUM` suffit donc à reconstruire le tableau complet sur le maître : pour chaque case, la somme vaut la valeur du seul processus qui l'a calculée.

```
MPI_Reduce(&local[0][0], &global[0][0], NB_TACHES * NB_CHAMPS,
    MPI_LONG_LONG, MPI_SUM, MAITRE, MPI_COMM_WORLD);
```

Le champ `CH_EXECUTEE` (mis à 1 par le processus qui exécute la tâche) est sommé comme les autres : sa valeur dans le tableau collecté doit être exactement 1, ce qui démontre que *chaque tâche a été exécutée une et une seule fois* (ni oubliée, ni dupliquée).

1.4 Vérification

Pour chaque tâche, le maître vérifie dans `verifier_resultats` :

- que la tâche a été exécutée exactement une fois (`CH_EXECUTEE` $= 1$) et que `generate_rsa_keys` a renvoyé 0 ;
- que $\gcd(e, \varphi) = 1$ et $(e \cdot d) \bmod \varphi = 1$, avec $\varphi = (p-1)(q-1)$ recalculé à partir des paires ;
- que (N, e, d) est identique au résultat d'un appel séquentiel de `generate_rsa_keys` sur la même paire (équivalence avec l'implémentation séquentielle).

1.5 Résultats

Le programme est exécuté avec 1, 2, 4 et 8 processus (`make run_all`). Pour les quatre nombres de processus, la sortie se termine par `Verification OK : 16 cles valides, identiques au sequentiel`. Les clés obtenues sont identiques d'une exécution à l'autre ; seul l'ordre d'affichage des lignes `[processus ...]` varie, car les processus écrivent en concurrence. Le tableau 1 donne les 16 clés et le processus qui a traité chaque tâche pour 4 processus.

Tâche	p	q	N	e	d	**Processus** ($P = 4$)
1	101	113	11413	3	7467	0
2	107	127	13589	5	10685	1
3	131	137	17947	3	11787	2
4	139	149	20711	5	4085	3
5	151	163	24613	7	6943	0
6	167	173	28891	3	19035	1
7	179	191	34189	3	22547	2
8	193	197	38021	5	15053	3
9	199	211	41989	13	6397	0
10	223	227	50621	5	20069	1
11	229	233	53357	5	42317	2
12	239	241	57599	11	20771	3
13	251	257	64507	3	42667	0
14	263	269	70747	3	46811	1
15	271	277	75067	7	42583	2
16	281	283	79523	11	35891	3

TABLE 1 – Clés RSA générées (identiques pour 1, 2, 4 et 8 processus)

À titre d'exemple, pour la tâche 1 : $\varphi = 100 \times 112 = 11200$, $e = 3$ (le plus petit impair premier avec φ, car $3 \nmid 11200$) et $3 \times 7467 = 22401 = 2 \times 11200 + 1$, donc $e \cdot d \equiv 1 \pmod{\varphi}$.

2 Partie III : évaluation des performances

2.1 Protocole de mesure

Le programme `bench_mpi.c` mesure les temps dans un seul lancement (`mpirun -oversubscribe -np 8`), les différents nombres de processus étant obtenus avec des sous-communicateurs (`MPI_Comm_split`) : pour $P \in \{1, 2, 4, 8\}$, seuls les P premiers processus participent. Le protocole est le suivant :

- **charge répétée** : chaque paire est traitée 100 000 fois, soit $16 \times 100\,000 = 1\,600\,000$ générations de clés (point 7 du sujet). Les tâches sont réparties de la même façon que dans `rsa_mpi.c` (distribution cyclique, la tâche t va au processus $t \bmod P$) ;
- **temps séquentiel** T_1 : mesuré par le maître seul, qui exécute les 1 600 000 tâches sans MPI (le sujet définit T_1 comme le temps séquentiel) ;
- **temps parallèle** T_P : mesuré avec `MPI_Wtime()` après un `MPI_Barrier`, collecte (`MPI_Reduce`) comprise ;
- **médiane de 5 mesures** pour chaque configuration, afin de limiter l'effet des aléas du système ;
- **contrôle de correction** : chaque processus calcule la somme $\sum(N+e+d)$ de ses tâches ; la somme globale doit être égale à celle de la version séquentielle (vérification réussie) ;
- les programmes sont compilés avec `-Wall -Wextra`, sans option d'optimisation.

Machine : Dell Inspiron 16, Ubuntu 24.04, OpenMPI, processeur Intel Core i7-11800H (11e génération, 1 socket, 8 cœurs physiques et 16 threads grâce à l'hyperthreading ; fréquence de base 2,3 GHz, jusqu'à 4,6 GHz en mode turbo). `nproc` renvoie 16.

2.2 Résultats

Processus P	**Temps** T_P **(ms)**	**Accélération** $S(P)$	**Efficacité** $E(P)$
Séquentiel	260,3	—	—
1	268,2	0,970	97,0 %
2	131,4	1,981	99,1 %
4	73,7	3,530	88,2 %
8	37,3	6,969	87,1 %

TABLE 2 – Temps, accélération et efficacité pour 1 600 000 générations de clés (médiane de 5 mesures)

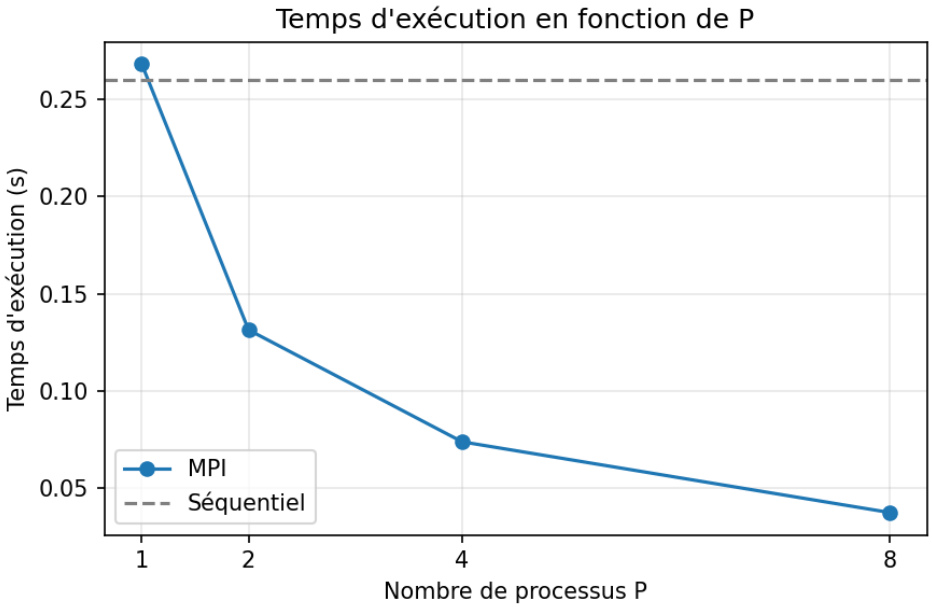


FIGURE 1 – Temps d'exécution en fonction du nombre de processus (la droite pointillée est le temps séquentiel)

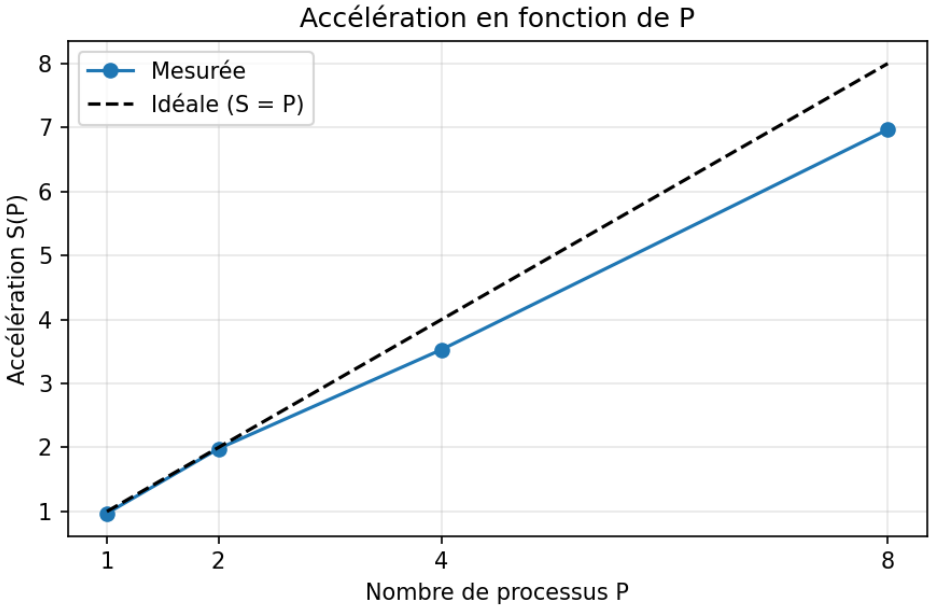


FIGURE 2 – Accélération mesurée et accélération idéale $S = P$

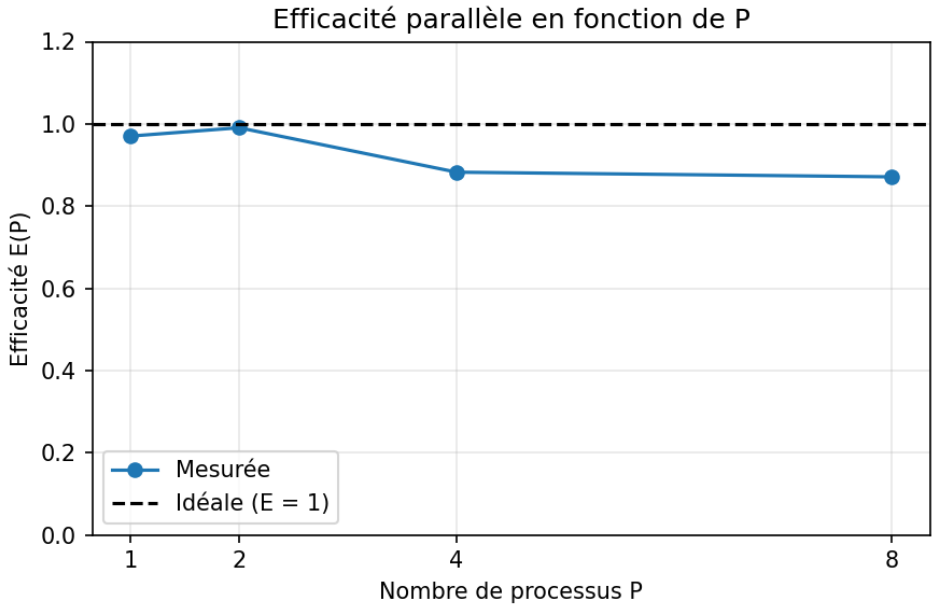


FIGURE 3 – Efficacité parallèle en fonction du nombre de processus

2.3 Analyse

Évolution du temps et de l'accélération. Le temps passe de 260,3 ms (séquentiel) à 131,4 ms, 73,7 ms et 37,3 ms pour 2, 4 et 8 processus : il diminue presque proportionnellement à P. L'accélération atteint 1,98 pour 2 processus, 3,53 pour 4 et 6,97 pour 8, soit une efficacité de 99,1 %, 88,2 % et 87,1 %. Le passage à l'échelle est donc bon mais pas parfait : l'efficacité est quasi idéale à 2 processus, puis chute de 11 points à 4 processus et reste ensuite stable jusqu'à 8.

Influence du nombre de tâches indépendantes. Les 16 tâches de l'énoncé sont indépendantes (aucune communication pendant le calcul), ce qui limite le parallélisme à 16 processus : au-delà, des processus resteraient sans tâche. Surtout, elles sont *très courtes* : en moyenne, une génération de clés coûte 0,2603/1 600 000 $\approx$ 163 ns avec ce protocole, soit environ 2,6 μs pour les 16 tâches. Exécuter l'énoncé tel quel ne permettrait donc aucune mesure significative, et le coût de démarrage de MPI (`MPI_Init`, lancement des processus, non mesuré ici mais bien supérieur à la microseconde) dominerait totalement le calcul, avec une accélération inférieure à 1. C'est la raison de la répétition de la charge (point 7 du sujet) : avec 1,6 million de tâches, le temps de calcul (260 ms) devient grand devant les coûts fixes.

Initialisation des processus. Le temps de `MPI_Init` et de lancement des processus n'est pas inclus dans les mesures, qui portent uniquement sur le calcul et la collecte (après `MPI_Barrier`). Il serait un surcoût fixe, indépendant de la charge : plus la charge répétée est grande, plus il devient négligeable.

Surcoût de communication. La seule communication est un `MPI_Reduce` d'un entier de 8 octets en fin de calcul (640 octets pour le tableau complet de `rsa_mpi.c`), plus un `MPI_Barrier` avant la mesure. Ce coût, de l'ordre de quelques dizaines de microsecondes

au plus, est négligeable devant des temps de plusieurs dizaines de millisecondes : il ne peut pas expliquer une perte d'efficacité de 12 %. De plus, l'estimation de la fraction séquentielle par la métrique de Karp–Flatt, $f = \dfrac{1/S - 1/P}{1 - 1/P}$, donne 0,9 % pour $P = 2$, 4,4 % pour $P = 4$ et 2,1 % pour $P = 8$: elle n'est pas constante, donc la perte d'efficacité n'est pas celle d'un surcoût séquentiel fixe au sens de la loi d'Amdahl.

Répartition de la charge. La distribution cyclique donne à chaque processus *le même nombre* de tâches, mais pas la même charge : le coût d'une génération dépend de la paire. L'exposant e est cherché par incréments de 2 à partir de 3 (on a $e = 3$ pour la tâche 1 mais $e = 13$ pour la tâche 9), et chaque essai coûte un calcul de `gcd` ; les tests de primalité de p et q et l'algorithme d'Euclide étendu varient aussi avec les valeurs. En comptant les divisions entières (opération dominante), les tâches coûtent entre 35 (tâche 1) et 73 (tâche 12) divisions, avec 50 en moyenne. Comme 16 est divisible par $P \in \{2, 4, 8\}$, le processus r traite toujours les mêmes paires ($i \equiv r \bmod P$), et le temps T_P est celui du processus le plus chargé. Le tableau 3 donne l'efficacité maximale qu'imposerait ce déséquilibre seul.

P	**Charge max.**	**Charge moy.**	E **maximale estimée**	E **mesurée**
2	413	399,5	96,7 %	99,1 %
4	233	199,8	85,7 %	88,2 %
8	119	99,9	83,9 %	87,1 %

TABLE 3 – Charge par processus (nombre de divisions entières pour un cycle de 16 tâches) et efficacité maximale due au seul déséquilibre

Cette estimation a la même forme que les mesures : efficacité élevée à 2 processus, chute à 4, puis quasi-stabilité entre 4 et 8. Le déséquilibre du coût des paires est donc la cause la plus probable de la perte d'efficacité. L'estimation reste toutefois approximative : elle compte les divisions sans tenir compte de leur durée réelle, et elle est pessimiste par rapport aux mesures (0,86 attendu contre 0,88 mesuré à 4 processus). Nous ne l'avons pas confirmée en mesurant le temps de chaque processus. Un autre facteur possible, non mesuré, est la fréquence turbo du processeur (jusqu'à 4,6 GHz) : elle diminue en général quand davantage de cœurs sont actifs, ce qui ralentit chaque processus lorsque P augmente.

Cas de $P = 1$. Avec un seul processus, l'efficacité est de 97 % : le temps MPI (268,2 ms) est supérieur de 3 % au temps séquentiel (260,3 ms), alors que le calcul est identique et que la barrière et la réduction sur un processus sont négligeables. Nous ne l'avons pas expliqué (fluctuation de mesure ou de fréquence du processeur ?). Si l'on prend le temps MPI à 1 processus comme référence, les accélérations deviennent 2,04, 3,64 et 7,18 (efficacités de 102 %, 91 % et 90 %) : cette référence change légèrement les valeurs, mais pas la forme de la courbe. Nous gardons la définition du sujet (T_1 séquentiel).

Limites de l'expérience.

— Les 8 processus de la mesure à $P = 8$ tiennent sur les 8 cœurs physiques, mais le placement n'est pas imposé (pas d'option `-bind-to core`) : le système peut placer deux processus sur les deux threads d'un même cœur, ce qui ralentirait chacun. Nous n'avons pas vérifié le placement.

— Pendant la mesure à $P < 8$, les processus inactifs attendent dans `MPI_Barrier`. Sous OpenMPI, cette attente est généralement active et peut occuper des cœurs ; ce biais éventuel n'a pas été vérifié.
— Les mesures dépendent de la charge choisie (100 000 répétitions) et du fait que le code est compilé sans optimisation.

Pistes d'amélioration. Le déséquilibre vient de l'alignement entre la distribution cyclique et la structure de 16 paires. Une distribution par blocs contigus sur la charge répétée donnerait à chaque processus toutes les paires en proportions égales, et pour les 16 tâches seules, un ordonnancement dynamique (maître–esclaves) équilibrerait mieux la charge. Ces variantes n'ont pas été testées.

Conclusion

— **Partie II** : `rsa_mpi.c` réutilise `generate_rsa_keys` sans modification, répartit les 16 tâches de façon cyclique (chaque tâche exécutée par exactement un processus), collecte les résultats par un `MPI_Reduce` et vérifie $\gcd(e, \varphi) = 1$, $(e \cdot d) \bmod \varphi = 1$ ainsi que l'identité avec le résultat séquentiel. La vérification est réussie pour 1, 2, 4 et 8 processus.
— **Partie III** : pour 1 600 000 générations de clés, le temps passe de 260,3 ms (séquentiel) à 37,3 ms avec 8 processus, soit une accélération de 6,97 et une efficacité de 87 %. L'efficacité est de 99 % à 2 processus et d'environ 88 % à 4 et 8 processus.
— La communication (un `MPI_Reduce` de quelques octets) est négligeable. La perte d'efficacité vient plus probablement du déséquilibre de coût entre les paires (p, q) que la distribution cyclique ne compense pas (estimation : 86 % à $P = 4$ et 84 % à $P = 8$), ce que nous n'avons pas confirmé par une mesure par processus.
— Les tâches de l'énoncé sont trop courtes (environ 2,6 μs au total) pour être mesurées ou accélérées par MPI : sans répétition de la charge, le coût de démarrage de MPI domine.